

Table des matières

INTRODUCTION	2
I. Généralités sur les arbres	3
1. Définition, représentation et propriétés	3
a. Définition.....	3
b. Représentation	3
c. Propriétés.....	3
2. Parcours	4
a. Parcours en profondeur.....	4
b. Parcours en largeur	4
c. Complexité	5
II. Arbres binaires et arbre binaires de recherche	5
1. Arbre binaire.....	5
a. Définition.....	5
b. Représentation	5
2. Arbre Binaire de Recherche (ABR)	5
a. Définition.....	5
b. Représentation	6
c. Propriétés.....	6
3. Opérations sur les arbres binaires de recherche.....	6
a. Recherche	6
b. Minimum et maximum	7
c. Successeur et Prédécesseur.....	7
d. Insertion.....	8
e. Suppression	9
4. Complexité	10
III. Arbres rouge-noir ou arbre bicolore	11
1. Présentation et définition d'un arbre rouge-noir.....	11
2. Hauteur d'un arbre rouge et noir	11
3. Rotations.....	12
4. Insertion.....	13
5. Suppression	15
6. Correction.....	18
7. Complexité	18
IV. Comparaison entre arbres binaires de recherche et rouge-noir	19
1. Comparaison.....	19
2. Domaines d'applications	19
CONCLUSION	20
Références bibliographiques et webographiques	21

INTRODUCTION

Les arbres sont des structures de données fondamentales en informatique très utilisés dans tous les domaines parce qu'ils sont bien adaptés à la représentation naturelle d'informations homogènes organisées et d'une grande commodité et rapidité de manipulation. De nombreux problèmes informatiques nécessitent de manipuler et d'organiser efficacement de grandes quantités de données. Dans ce contexte, les arbres binaires de recherche et les arbres rouge-noir se sont révélés être des structures de données puissantes et polyvalentes.

Ainsi dit, quels avantages présentent l'utilisation d'un arbre binaire de recherche rouge-noir par rapport à un arbre binaire de recherche classique pour le stockage et la recherche de données ? Dans ce travail, nous avons plongé dans le monde des arbres binaires de recherche et des arbres rouge-noir, deux concepts clés de l'informatique.

Vous découvrirez comment ces structures de données peuvent résoudre efficacement des problèmes de recherche, d'insertion et de suppression de données, et comment elles répondent à des exigences spécifiques telles que la gestion efficace de l'équilibre de l'arbre. Tout d'abord, nous explorerons les bases des arbres binaires de recherche. Ensuite, nous nous plongerons dans les propriétés et les caractéristiques des arbres rouge-noir. Ces arbres étendent les fonctionnalités des arbres binaires de recherche en imposant des règles spécifiques pour l'équilibrage optimal de la structure de données. Enfin, nous comparerons de chaque structure, ainsi donc, présenter les différents domaines d'application de ces derniers.

I. Généralités sur les arbres

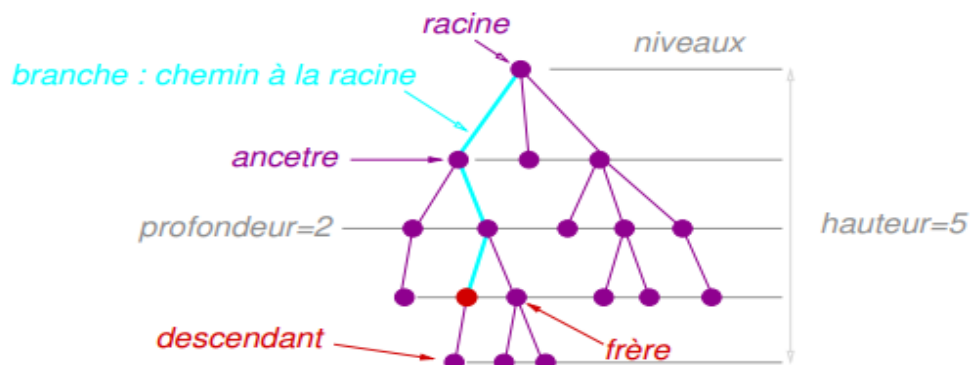
1. Définition, représentation et propriétés

a. Définition

L'arbre est une structure de donnée qui généralise la liste : alors qu'une cellule de liste a un seul successeur (leur suivant), dans un arbre il peut y en avoir plusieurs. On parle alors de nœud (au lieu de cellule). Un nœud père peut avoir plusieurs nœuds fils. Un fils n'a qu'un seul père, et tous les nœuds ont un ancêtre commun appelé la racine de l'arbre (le seul nœud qui n'a pas de père).

b. Représentation

On indique ici une représentation par pointeur. Il en existe d'autres, par pointeurs, ou des représentations par tableaux. Un dessin pour aider à comprendre la représentation des arbres par pointeur fils / pointeur frère :



Quand n a deux fils, son fils gauche est $n.fils$ et son fils droit est $n.fils.frère$. Dans toutes les représentations.

c. Propriétés

☞ Taille d'un arbre :

- Nombre de nœuds de l'arbre ;
- Définition récursive ;
- $taille(arbre) = 1 + somme(taille(Sa))$ pour tout Sa , sous arbre de la racine de l'arbre.

☞ Branche :

- Chemin d'une feuille ;
- Autant de branches que de feuilles.

☞ Hauteur d'un arbre :

- La plus grande profondeur de l'ensemble des nœuds de l'arbre ;
- La longueur de la branche la plus longue ;

- Hauteur d'un arbre vide = -1

☞ **Arbre de degré n** : arbre dont tous les nœuds sont au maximum de degré N

- Cas particulier 1 : arbre binaire (arbre de degré 2) ;
- Cas particulier 2 : arbres dégénérés ;
- Arbres de degré 1 ;
- Ce sont en fait des listes ;
- Attention : en général, un arbre dit n -aire est un arbre dont le degré n'est pas connu.

☞ **Niveau d'un arbre** :

- Ensemble des nœuds de même profondeur ;
- Valeur du niveau = valeur de la profondeur des nœuds.

2. Parcours

a. Parcours en profondeur

Dans un parcours en profondeur d'abord, on descend le plus profondément possible dans l'arbre puis, une fois qu'une feuille a été atteinte, on remonte pour explorer les autres branches en commençant par la branche « la plus basse » parmi celles non encore parcourues ; les fils d'un nœud sont bien évidemment parcourus suivant l'ordre sur l'arbre.

PP(A)

Si A n'est pas réduit à une feuille **faire**

Pour tous les fils u de $\text{racine}(A)$ **faire dans l'ordre PP**(u)

Les parcours permettent d'effectuer tout un ensemble de traitement sur les arbres. La figure ci-dessous présente trois algorithmes qui affichent les valeurs contenues dans les nœuds d'un arbre binaire, suivant des parcours en profondeur préfixe, infixé et postfixé, respectivement.

PRÉFIXE(A)	INFIXE(A)	POSTFIXE(A)
si A $\neq$ NIL alors	si A $\neq$ NIL alors	si A $\neq$ NIL alors
<i>affiche</i> $\text{racine}(A)$	INFIXE(FILS-GAUCHE(A))	POSTFIXE(FILS-GAUCHE(A))
PRÉFIXE(FILS-GAUCHE(A))	<i>affiche</i> $\text{racine}(A)$	POSTFIXE(FILS-DROIT(A))
PRÉFIXE(FILS-DROIT(A))	INFIXE(FILS-DROIT(A))	<i>affiche</i> $\text{racine}(A)$

b. Parcours en largeur

Dans un parcours en largeur d'abord, tous les nœuds à une profondeur i doivent avoir été visités avant que le premier nœud à la profondeur $i + 1$ ne soit visité. Un tel parcours nécessite que l'on se souvienne de l'ensemble des branches qu'il reste à visiter. Pour ce faire, on utilise une file (ici notée F).

PL(A)

$F \leftarrow \{\text{racine}(A)\}$

tant que $F \neq 0$ **faire**

$u \leftarrow \text{SUPPRESSION}(F)$

Pour tous les fils v de u **faire dans l'ordre** INSERTION (F, v)

c. Complexité

La complexité de la recherche (tant en profondeur qu'en largeur) vaut **n** car lors d'un parcours d'un arbre de n nœuds tous les nœuds sont traités. Un arbre binaire ne permet donc de gagner du temps dans la recherche que si l'on dispose à priori d'un indice sur la position de l'élément.

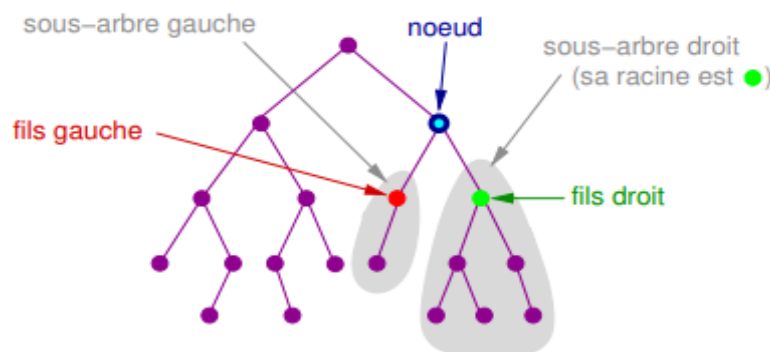
II. Arbres binaires et arbre binaires de recherche

1. Arbre binaire

a. Définition

Un **arbre binaire** est une structure de données qui peut se représenter sous la forme d'une hiérarchie dont chaque élément est appelé nœud, le nœud initial étant appelé racine. Dans un arbre binaire, chaque élément possède au plus deux éléments appelés "fils" au niveau inférieur, habituellement appelés gauche et droit. Du point de vue de ces éléments fils, l'élément dont ils sont issus au niveau supérieur racine. Au niveau directement inférieur, il y a au plus deux nœuds fils. En continuant à descendre aux niveaux inférieurs, on peut en avoir quatre, puis huit, seize, etc. c'est à dire la suite des puissances de deux. Un nœud n'ayant aucun fils est appelé feuille. La hauteur d'un arbre est le nombre de niveaux total, commençant par 0 ; autrement dit la distance entre la feuille la plus éloignée et la racine. Le niveau où se trouve un nœud particulier est appelé profondeur de ce nœud.

b. Représentation



Chaque nœud a potentiellement un fils gauche et un fils droit.

Un arbre binaire de **hauteur h** contient au plus $2^{h+1} - 1$ nœuds (avec la convention : hauteur = nombre de niveaux)

2. Arbre Binaire de Recherche (ABR)

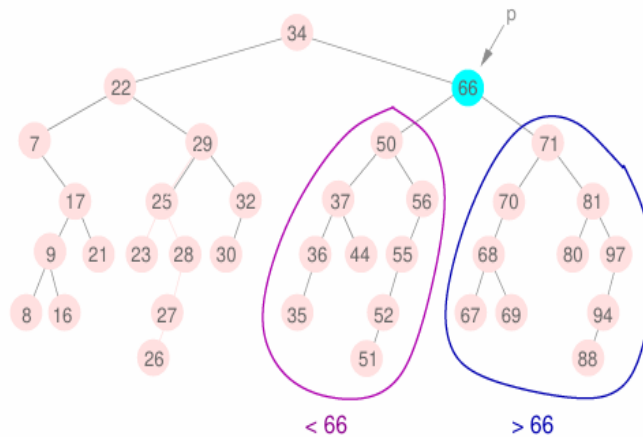
a. Définition

Un **arbre binaire de recherche** est un arbre binaire dans lequel chaque nœud possède une clé, telle que chaque nœud du sous arbre gauche ait une clé inférieure ou égale à celle du nœud considéré, et que chaque nœud du sous arbre droit possède une clé supérieure ou égale à celle-ci ; Selon la mise en œuvre de l'ABR, on pourra interdire ou non des clés de valeur égale. Les nœuds que l'on ajoute deviennent des feuilles de l'arbre. Un arbre binaire de recherche est dit équilibré si la différence de profondeur de deux feuilles quelconques est d'au plus 1. Il existe

de nombreux types d'arbres binaires de recherche, parmi lesquels les arbres rouge-noir et les arbres binaires de recherche optimaux (dont nous prendrons le soin de bien étudier).

b. Représentation

Ci-dessous est un exemple d'arbre binaire de recherche.



c. Propriétés

- **Relation d'ordre** : Les nœuds d'un ABR sont organisés de telle manière que pour chaque nœud, toutes les valeurs dans son sous arbre gauche sont inférieures à sa valeur, et toutes les valeurs dans son sous arbre droit sont supérieures à sa valeur. Cette propriété est appelée la propriété de recherche.
- **Recherche efficace** : En raison de la propriété de recherche, la recherche d'une valeur spécifique dans un ABR peut être réalisée efficacement en effectuant une comparaison à chaque nœud et en descendant dans le sous arbre approprié.
- **Insertion ordonnée** : L'insertion d'une nouvelle valeur dans un ABR est réalisée en respectant la propriété de recherche. La valeur est insérée à sa position appropriée dans l'arbre de manière à maintenir l'ordre.
- **Suppression ordonnée** : La suppression d'une valeur dans un ABR nécessite de préserver la propriété de recherche. Selon les cas, le nœud peut être supprimé directement ou remplacé par son successeur ou son prédécesseur dans l'ordre de recherche.
- **Parcours ordonné (infixe)** : Un parcours ordonné d'un ABR visite tous les nœuds de l'arbre dans l'ordre croissant des valeurs. Cela permet d'obtenir une séquence triée des valeurs stockées dans l'arbre.
- **Complexité temporelle** : Dans un ABR équilibré, la recherche, l'insertion et la suppression ont une complexité temporelle moyenne de $O(\log n)$, où n est le nombre de nœuds dans l'arbre. Cependant, dans le pire des cas, lorsque l'arbre est déséquilibré, ces opérations peuvent avoir une complexité de $O(n)$, ce qui peut être évité en utilisant des variantes d'ABR équilibrés comme les arbres rouges-noirs.

3. Operations sur les arbres binaires de recherche

Un arbre binaire de recherche de hauteur h permet de mettre en œuvre les opérations fondamentales d'ensemble dynamique, telles RECHERCHER, PRÉDÉCESSEUR, SUCCESSEUR, MINIMUM, MAXIMUM, INSÉRER et SUPPRIMER, avec un temps $O(h)$.

a. Recherche

La recherche d'un nœud ayant une clé particulière dans un arbre binaire peut se faire de manière : récursive et itérative.

Pour faire une recherche, on commence par examiner la racine, si sa clé est la clé recherchée, l'algorithme se termine et renvoie la racine. Si elle est strictement inférieure, alors elle est dans le sous arbre gauche, sur lequel on effectue alors récursivement la recherche. De même si la clé recherchée est strictement supérieure à la clé de la racine, la recherche continue dans le sous arbre droit. Si on atteint une feuille dont la clé n'est pas celle recherchée, on sait alors que la clé recherchée n'appartient à aucun nœud, elle ne figure donc pas dans l'arbre de recherche. L'algorithme itératif de l'opération rechercher dans l'arbre binaire de recherche peut se donner comme suit

fonction RECHERCHE(x, p)

out : $\square$ si x n'apparaît pas dans le sous-arbre enraciné en p ,
le nœud de valeur x sinon,

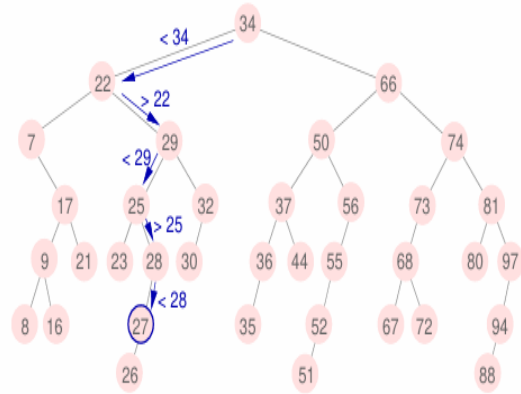
si $p = \square$ alors renvoyer $\square$,

si $val(p) = x$ alors renvoyer p ,

si $val(p) > x$ alors renvoyer RECHERCHE($x, filsG(p)$),

renvoyer RECHERCHE($x, filsD(p)$),

fin fonction



b. Minimum et maximum

On peut toujours trouver un élément d'un arbre binaire de recherche dont la clé est un minimum en suivant les pointeurs gauches à partir de la racine jusqu'à ce qu'on rencontre NIL. La procédure suivante retourne un pointeur sur l'élément minimal du sous arbre enraciné au nœud x .

Fonction maximum (A : arbre) : arbre

Début

Si ($A \rightarrow FD \neq Nil$) alors

Retourner (maximum ($A \rightarrow FD$))

Sinon

Retourner (A)

Finsi

Fin

Fonction minimum (A : arbre) : arbre

Début

Si ($A \rightarrow FG \neq Nil$) alors

Retourner (minimum ($A \rightarrow FG$))

Sinon

Retourner (A)

Finsi

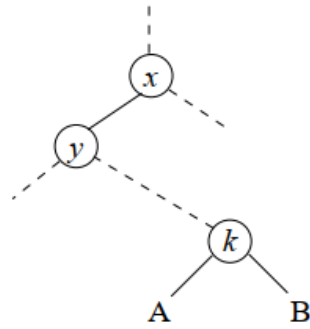
Fin

Le pseudo code pour recherche le maximum dans un arbre binaire de recherche est symétrique.

c. Successeur et Prédécesseur

Dans un arbre binaire de recherche, le successeur d'un nœud x est le nœud contenant la plus petite clé supérieure à x , et son prédécesseur est le nœud contenant la plus grande clé inférieure à x . La procédure suivante retourne le successeur d'un nœud x dans un arbre binaire de recherche, s'il existe, et NIL si x possède la plus grande clé de l'arbre.

Si toutes les clés dans l'arbre sont distinctes, le successeur d'un nœud x est le nœud contenant la plus petite clé supérieure à x .



Considérons le fragment d'arbre présenté à la figure ci-dessous. De par la propriété des arbres binaires de recherche :

- Le sous arbre A ne contient que des clés inférieures à k et ne peut contenir le successeur de k .
- Le sous arbre B ne contient que des clés supérieures à k et peut contenir le successeur de k s'il n'est pas vide.
- y désigne le plus proche ancêtre de k qui soit le fils gauche de son père ($y = k$ si k est fils gauche de son père). Tous les ancêtres de k jusqu'à y sont inférieurs ou égaux à k et leurs sous arbres gauches ne contiennent que des valeurs inférieures à k . x est le père de y . Sa valeur est supérieure à toutes celles contenues dans son sous arbre gauche (de racine y) et donc à k et à celles de B . Toutes les valeurs de son sous arbre droit sont supérieures à x .

En résumé, si B est non vide, son minimum est le successeur de k , sinon le successeur de k est le premier ancêtre de k dont le fils gauche est aussi ancêtre de k .

```

ARBRE-SUCCESEUR(x)
Si droit(x) ≠ NIL alors
    Renvoyer ARBRE-MINIMUM(droit(x))
y ← père(x)
Tant que y ≠ NIL et x = droit(y) faire
    x ← y
y ← père(x)
Renvoyer y

```

d. Insertion

L'insertion d'un nœud commence par une recherche : on cherche la clé du nœud à insérer ; lorsqu'on arrive à une feuille, on ajoute le nœud comme fils de la feuille en comparant sa clé à celle de la feuille : si elle est inférieure, le nouveau nœud sera à gauche ; sinon il sera à droite. La procédure suivante montre comment insérer un élément dans un arbre binaire de recherche.


```

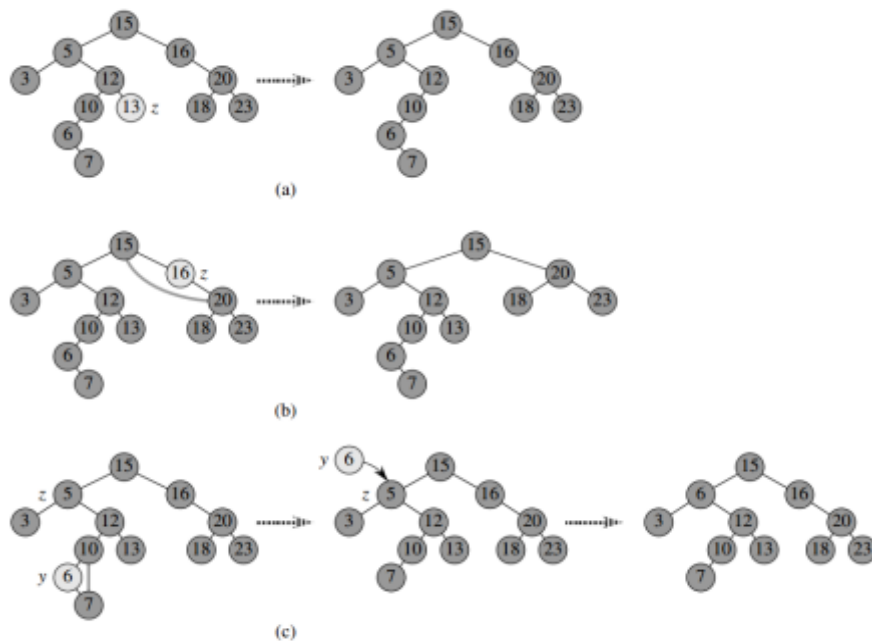
Procédure Insérer_rec (var A : arbre, e : entier)
Var P : arbre
Début
    Si (A = Nil) alors
        Allouer (A)
        A-> Val ← e
        A-> FG ← Nil
        A-> FD ← Nil
    Sinon
        Si (A-> val > e) alors
            Insérer_rec(A-> FG, e)
        Sinon
            Insérer_rec (A-> FD, e)
    Finsi
Finsi
Fin

```

e. Suppression

Plusieurs cas sont à considérer, une fois que le nœud à supprimer a été trouvé à partir de sa clé :

- **Suppression d'une feuille** : Il suffit de l'enlever de l'arbre vu qu'elle n'a pas de fils.
- **Suppression d'un nœud avec un enfant** : Il faut l'enlever de l'arbre en le remplaçant par son fils.
- **Suppression d'un nœud avec deux enfants** : le nœud à supprimer a deux fils, on le remplace par son successeur (qui, dans ce cas, est toujours le minimum de son fils droit).



Le pseudo code suivant montre les différents cas de suppression dans un arbre binaire de recherche.

```

fonction SUPPRIMER( $x, p$ )
  si  $val(p) > x$  alors
     $filG(p) := SUPPRIMER(x, filG(p))$ ,
  sinon si  $val(p) < x$  alors
     $filD(p) := SUPPRIMER(x, filD(p))$ ,
  sinon si  $filG(p) = \square$  alors
    renvoyer  $filD(p)$ ,
  sinon si  $filD(p) = \square$  alors
    renvoyer  $filG(p)$ ,
  sinon
     $val(p) := GETMIN(filD(p))$ ,
     $filD(p) := SUPPRIMER(val(p), filD(p))$ ,
  renvoyer  $p$ ,
fin fonction

```

4. Complexité

Si h est la hauteur de l'arbre, on peut aisément montrer que tous les algorithmes précédents ont une complexité en $O(h)$.

Démonstration :

Dans le cas de la recherche d'un élément x dans un arbre binaire de recherche de taille n nous avons deux cas :

- Si la valeur de x est inférieure à la valeur de l'élément à la racine alors le nœud x se trouve dans le sous arbre gauche ;
- Sinon le nœud x se trouve dans le sous arbre droit.

En fait nous constatons que pour retrouver le nœud x on prend un problème de taille n (l'arbre) puis on divise en deux sous problème de taille $n/2$ ceci nous amène à l'équation de récurrence : $T(n) = T(n/2) + c$ avec $T(n)$ la complexité du problème En résolvant cette équation par la méthode de substitution on obtient :

$$\begin{aligned}
 T(n) &= T\left(\frac{n}{2}\right) + c \\
 &= T\left(\frac{n}{2^2}\right) + 2c \\
 &\dots \\
 &= T\left(\frac{n}{2^h}\right) + hc
 \end{aligned}
 \qquad
 \begin{aligned}
 T\left(\frac{n}{2^h}\right) &= O(1) \\
 \rightarrow \frac{n}{2^h} &= 1 \\
 \rightarrow n &= 2^h \\
 \rightarrow \log_2 n &= h
 \end{aligned}$$

Nous avons montré qu'un arbre binaire de recherche de hauteur h permettait de mettre en œuvre les opérations de fondamentales d'ensemble dynamique, telles que *rechercher*, *maximum*, *minimum*, *prédécesseur*, *successeur*, *insérer* et *supprimer* avec un temps $O(h)$. Les opérations sont donc d'autant plus rapides que la hauteur de l'arbre est petite, mais si cette hauteur est grande, les performances risquent de ne pas être meilleures qu'avec les listes chaînées (Ici si les données suivent une distribution linéaire nous aurons ici une complexité en $O(n)$). Les

arbres rouges et noirs sont l'un des nombreux modèles d'arbres de recherche équilibrés, qui permettent aux opérations d'ensemble dynamique de s'exécuter en temps $O(\log_2(n))$ même dans le cas le plus défavorable.

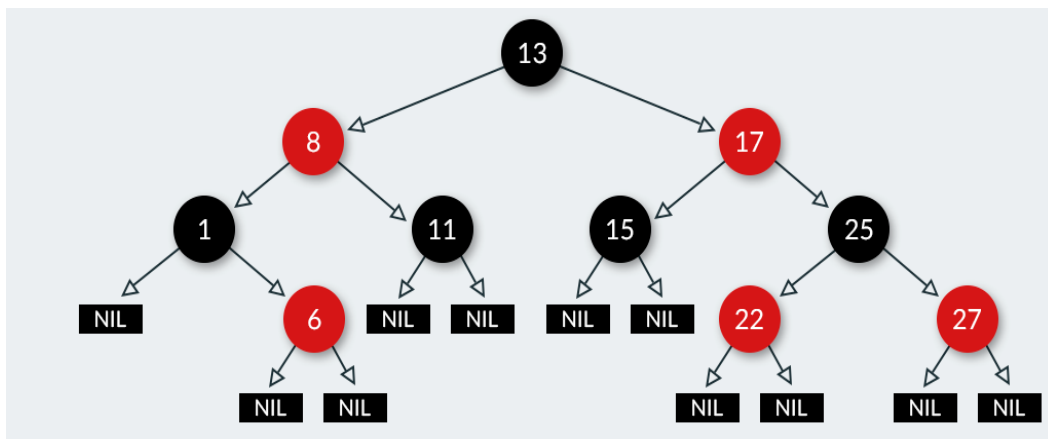
III. Arbres rouge-noir ou arbre bicolore

1. Présentation et définition d'un arbre rouge-noir

Un **arbre rouge-noir** (ARN) est un arbre binaire de recherche satisfaisant les cinq propriétés suivantes :

1. Chaque nœud est soit rouge, soit noir.
2. La racine est noire.
3. Chaque feuille (NIL) est noire.
4. Si un nœud est rouge, alors ses deux fils sont noirs.
5. Tous les chemins descendants reliant un nœud donné à une feuille (du sous arbre dont il est la racine) contiennent le même nombre de nœuds noirs.

. La figure ci-dessous présente un exemple d'arbre rouge et noir.



2. Hauteur d'un arbre rouge et noir

On appelle **hauteur noire** d'un nœud x , noté $hn(x)$, le nombre de nœuds noirs présents dans un chemin quelconque descendant du nœud x , à une feuille, sans que le nœud x ne soit y compris.

Les opérations de recherche dans un arbre binaire de recherche sont de l'ordre de $O(h)$ avec h étant la hauteur de l'arbre. En considérant sa hauteur, le lemme suivant nous montre pourquoi les arbres rouge-noir sont de bons arbres de recherche.

Lemme : Un arbre rouge et noir comportant n nœuds internes à une hauteur au plus égale à $2 \cdot \log_2(n + 1)$.

Démonstration :

- ❖ Commençons par montrer que le sous arbre enraciné en un nœud x quelconque contient au moins $2^{hn(x)} - 1$ nœuds internes.

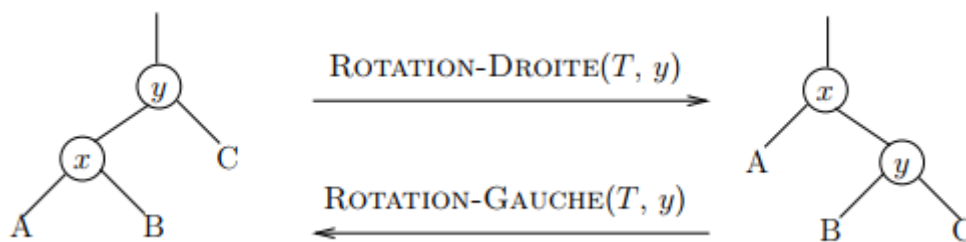
Cette affirmation peut se démontrer par la récurrence sur la hauteur de x :

- Si la *hauteur* de $x = 0$, alors x est une feuille et le sous arbre enraciné en x contient effectivement au moins $2^{hn(x)} - 1 = 2^0 - 1 = 0$ nœuds internes.

- Pour l'étape inductive, soit un nœud interne x de hauteur positive ayant deux enfants. Chaque enfant a une hauteur noire $hn(x)$ ou $hn(x) - 1$, selon que sa couleur est rouge ou noire. Comme la hauteur d'un enfant de x est inférieure à celle de x lui-même, on peut appliquer l'hypothèse de récurrence pour conclure que chaque enfant a au moins $2^{hn(x)-1} - 1$ nœuds internes.
- Le sous arbre de racine x contient donc au moins $(2^{hn(x)-1} - 1) + (2^{hn(x)-1} - 1) = 2^{hn(x)} - 1$ nœuds internes, ce qui démontre l'assertion.
- D'après la propriété 4 sur les arbres rouge-noir, au moins la moitié des nœuds d'un chemin simple reliant la racine à une feuille, racine non comprise, doivent être noirs. En conséquence, la hauteur noire de la racine doit valoir au moins $\frac{h}{2}$; donc, $n \geq 2^{h/2} - 1$. En faisant passer le 1 dans le membre gauche et en prenant le logarithme des deux membres, on obtient : $\log(n + 1) \geq \frac{h}{2}$; soit $h \leq 2 \log(n + 1)$.

3. Rotations

Lorsqu'on exécute les opérations ARBRE-INSÉRER et ARBRE-SUPPRIMER sur un arbre rouge-noir à n clés (Nous verrons ces procédures plus bas), prennent un temps $O(\lg n)$. Comme elles modifient l'arbre, le résultat pourrait violer les propriétés d'arbre rouge-noir énumérées plus haut. Pour restaurer ces propriétés, il faut changer les couleurs de certains nœuds de l'arbre et également modifier la chaîne des pointeurs. Pour retrouver les propriétés de l'arbre rouge-noir, nous utilisons un algorithme de rotation. **Les rotations préservent la propriété des arbres de recherche mais pas la propriété des arbres.**



Le pseudo code de la rotation gauche pour un arbre binaire T et un nœud x (la rotation droite étant symétrique) est le suivant :

```

RotationGauche(T,x)
Début
  y <- droit(x)
  droit(x) <- gauche(y)
  Si gauche(y) <> Nil Alors
    père(gauche(y)) <- x
  FinSi
  père(y) <- père(x)
  Si père(y) = Nil Alors
    racine(T) <- y
  Sinon Si x = gauche(père(x)) Alors
    gauche(père(x)) <- y
  Sinon
    droit(père(x)) <- y
  FinSi
  gauche(y) <- x
  père(x) <- y
Fin

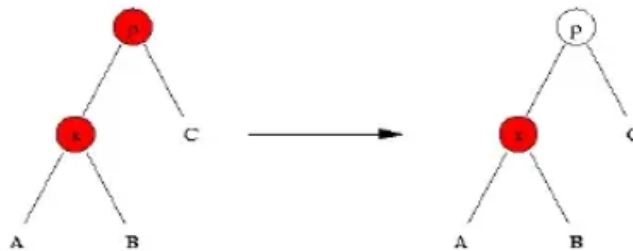
```

4. Insertion

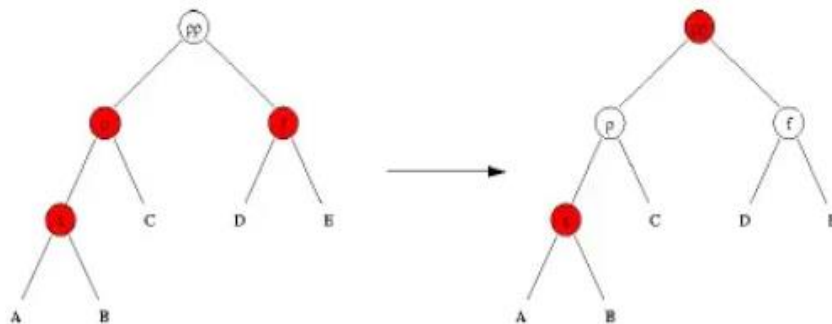
L'insertion d'une valeur dans un arbre rouge et noire commence par l'insertion usuelle d'une valeur dans un arbre binaire de recherche. Le nouveau nœud devient **rouge** de telle sorte que la *propriété (3)* reste vérifiée. Par contre, la *propriété (2)* n'est plus nécessairement vérifiée. Si le père du nouveau nœud est également rouge, la *propriété (2)* est violée. Afin de rétablir la propriété (2), l'algorithme effectue des modifications dans l'arbre à l'aide de rotations. Ces modifications ont pour but de rééquilibrer l'arbre.

Soit x le nœud et p son père qui sont tous les deux rouges. L'algorithme distingue plusieurs cas.

Cas 0 : le nœud père p est la racine de l'arbre. Le nœud père devient alors noir. La propriété (2) est maintenant vérifiée et la propriété (3) le reste. C'est le seul cas où la hauteur noire de l'arbre augmente.

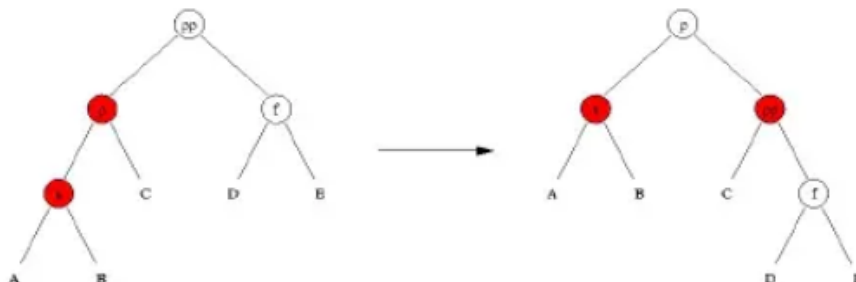


Cas 1 : le frère f de p (l'oncle de x) est rouge. Les nœuds p et f deviennent noirs et leur père pp devient rouge. La propriété (3) reste vérifiée mais la propriété ne l'est pas nécessairement si le père de pp est aussi rouge. Par contre, l'emplacement des deux nœuds rouges consécutifs s'est déplacé vers la racine.

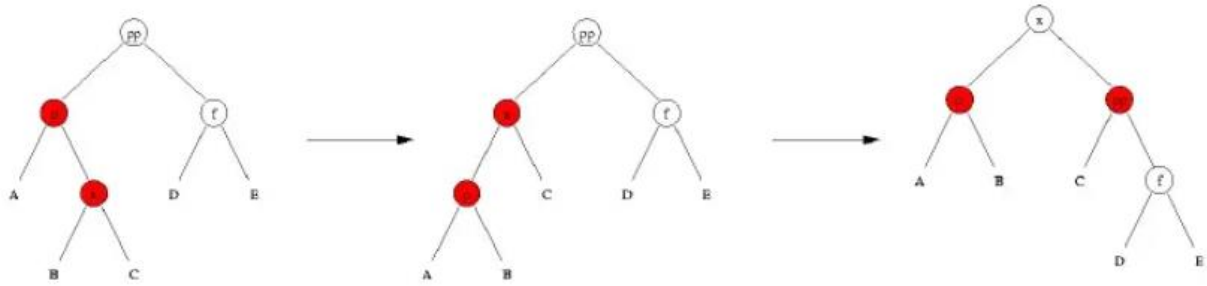


Cas 2 : le frère f de p (l'oncle de x) est noir. Par symétrie on suppose que p est le fils gauche de son père. L'algorithme distingue à nouveau deux cas suivant que x est le fils gauche ou le fils droit de p .

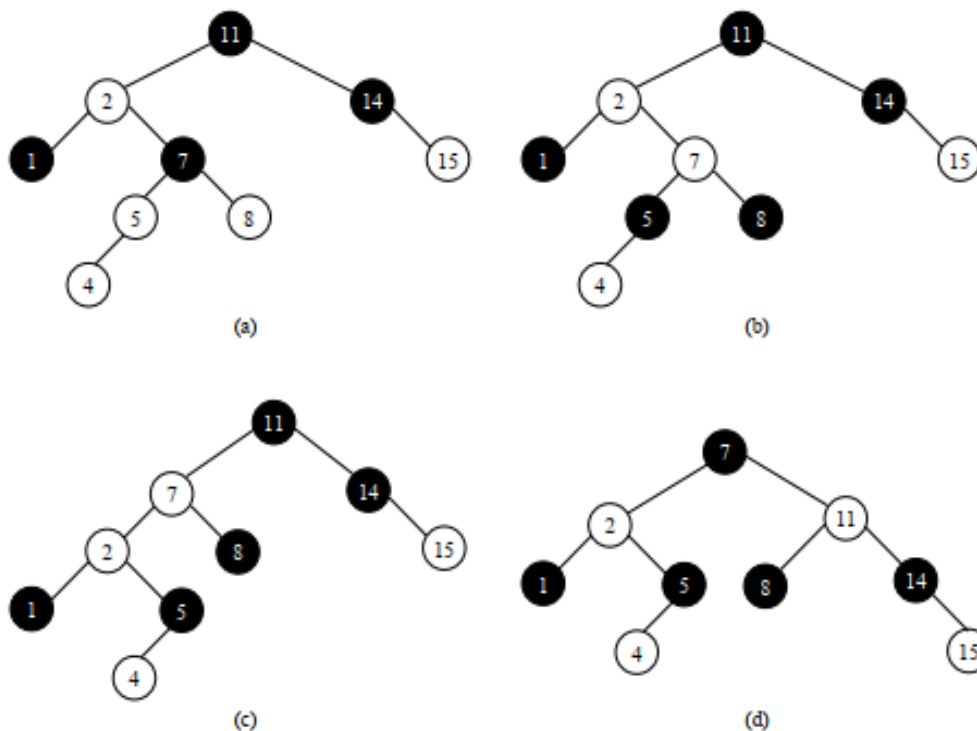
Cas 2a : x est le fils gauche de p . L'algorithme effectue une rotation droite entre p et pp . Ensuite le nœud p devient noir et le nœud pp devient rouge. L'algorithme s'arrête alors puisque les propriétés (2) et (3) sont maintenant vérifiées.



Cas 2b : x est le fils droit de p . L'algorithme effectue une rotation gauche entre x et p de sorte que p deviennent le fils gauche de x . On est ramené au cas précédent et l'algorithme effectue une rotation droite entre x et pp . Ensuite le nœud x devient noir et le nœud pp devient rouge. L'algorithme s'arrête alors puisque les propriétés (2) et (3) sont maintenant vérifiées.



❖ Exemple d'insertion dans un ARN



- (a) L'oncle de x (qui vaut 4) est également rouge : on se trouve donc dans le [Cas pathologique1]
- (b) La couleur noire du grand-père descend sur le père et l'oncle de x , lequel est repeint en rouge, puis le problème est remonté de deux crans : ici x vaut 7.
- (c) L'oncle de x est noir et x est le fils droit de son père : on se trouve donc dans le [Cas pathologique 2(a)]. On applique alors une rotation gauche sur le père de x et on aboutit à la situation du cas suivant : ici x vaut 2.
- (d) L'oncle de x est noir, x est le fils gauche de son père et son père est le fils gauche de son grand-père : on se trouve donc dans le [Cas pathologique 2(b)]. On applique alors une rotation droite sur le grand-père de x . Le père de x est alors repeint en noir et l'ancien grand-père de x en rouge, ce qui nous donne un arbre rouge et noir valide.

❖ Algorithme d'insertion et de correction dans un ARN

```

RNInsertion(T,x)
Début
  ABRInsertion(T,x)
  couleur(x) <- Rouge
  TantQue x <> racine(T) et couleur(père(x)) = Rouge Faire
    Si père(x) = gauche(grand-père(x)) Alors
      y <- droit(grand-père(x))
      Si couleur(y) = Rouge Alors
        // Pathos 1
        couleur(père(x)) <- Noir
        couleur(y) <- Noir
        couleur(grand-père(x)) <- Rouge
        x <- grand-père(x)
      Sinon
        Si x = droit(père(x)) Alors
          // Pathos 2(a)
          x <- père(x)
          RotationGauche(T,x)
        FinSi
        // Pathos 2(b)
        couleur(père(x)) <- Noir
        couleur(grand-père(x)) <- Rouge
        RotationDroite(T, grand-père(x))
      FinSi
    Sinon
      (même chose que précédemment en échangeant droit et gauche)
  FinSi
FinTantQue
couleur(racine(T)) <- Noir
Fin

```

5. Suppression

Comme pour l'insertion d'une valeur, la suppression d'une valeur dans un arbre rouge et noir commence par supprimer un nœud comme dans un arbre binaire de recherche. Si le nœud qui porte la valeur à supprimer a *zéro ou un fils*, c'est ce nœud qui est supprimé et son éventuel fils prend sa place. Si au contraire, ce nœud a *deux fils*, il n'est pas supprimé. La valeur qu'il porte est remplacée par la valeur suivante dans l'ordre et c'est le nœud qui portait cette valeur suivante qui est supprimé. Ce nœud supprimé est le nœud au bout de la branche gauche du sous-arbre droit du nœud qui portait la valeur à supprimer. Ce nœud n'a pas de fils gauche.

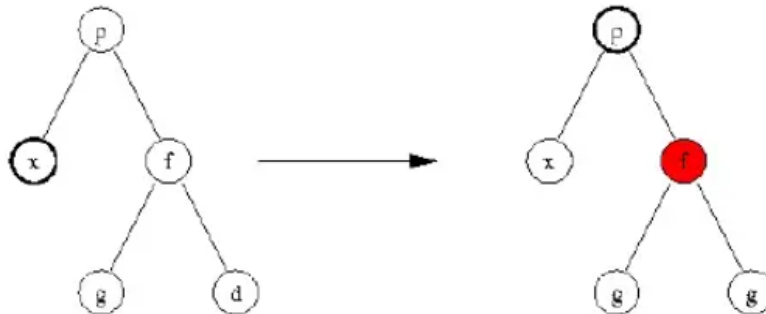
Si le nœud supprimé est **rouge**, la propriété (3) reste vérifiée. Si le nœud supprimé est **noir**, on considère que le nœud qui remplace le nœud supprimé porte une **couleur noire** en plus. Ceci signifie qu'il devient **noir** s'il est **rouge** et qu'il devient **doublement noir** s'il est déjà **noir**. La propriété (3) reste ainsi vérifiée mais il y a éventuellement un nœud qui est doublement noir. Afin de supprimer ce nœud doublement noir, l'algorithme effectue des modifications dans l'arbre à l'aide de rotation. Soit x le nœud doublement noir.

Cas 0 : le nœud x est la racine de l'arbre. Le nœud x devient simplement noir. La propriété (2) est maintenant vérifiée et la propriété (3) le reste. C'est le seul cas où la hauteur noire de l'arbre diminue.

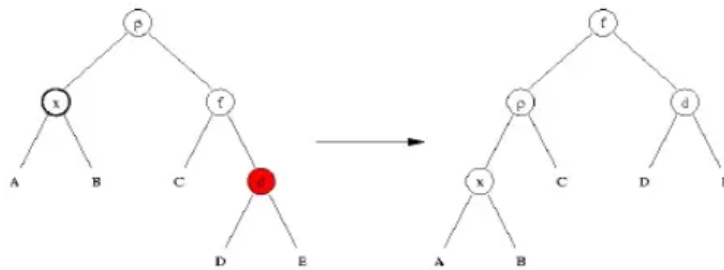


Cas 1 : le frère f de x est noir. Par symétrie, on suppose que x est le fils gauche de son père p et que f est donc le fils droit de p . Soient g et d les fils gauche et droit de f . L'algorithme distingue à nouveau trois cas suivants les couleurs des nœuds g et d .

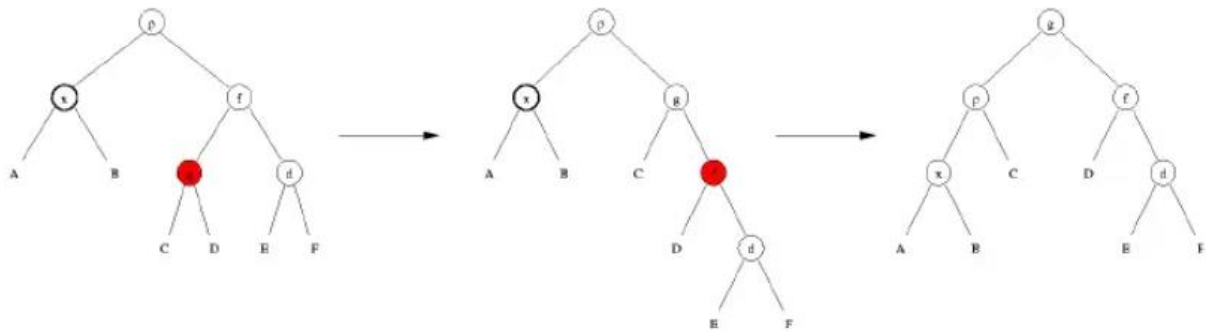
Cas 1a : les deux fils g et d de f sont noirs. Le nœud x devient noir et le nœud f devient rouge. Le nœud p porte une couleur noire en plus. Il devient noir s'il est rouge et il devient doublement noir s'il est déjà noir. Dans ce dernier cas, il reste encore un nœud doublement noir mais il s'est déplacé vers la racine de l'arbre. C'est ce dernier cas qui est représenté à la figure ci-dessous.



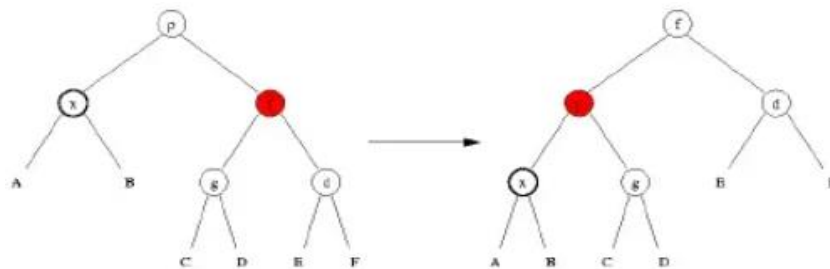
Cas 1b : le fils droit d de f est rouge. L'algorithme effectue une rotation gauche entre p et f . Le nœud f prend la couleur du nœud p . Les nœuds x, p et d deviennent noirs et l'algorithme se termine.



Cas 1c : le fils droit d est noir et le fils gauche g est rouge. L'algorithme effectue une rotation droite entre f et g . Le nœud g devient noir et le nœud f devient rouge. Il n'y a pas deux nœuds rouges consécutifs puisque la racine du sous-arbre D est noire. On est ramené au cas précédent puisque maintenant, le frère de x est g qui est noir et les deux fils de g sont noirs et rouges. L'algorithme effectue alors une rotation entre p et g . Le nœud f redevient noir et l'algorithme se termine.



Cas 2 : le frère f de x est rouge. Par symétrie, on suppose que x est le fils gauche de son père p et que f est donc le fils droit de p . Puisque f est rouge, le père p de f ainsi que ses deux fils g et d sont noirs. L'algorithme effectue alors une rotation gauche entre p et f . Ensuite p devient rouge et f devient noir. Le nœud x reste doublement noir mais son frère est maintenant le nœud g qui est noir. On est donc ramené au cas 1.



❖ Algorithme de suppression et de correction dans un ARN

```

RNSuppression(T,x)
Début
  Si gauche(x) = Nil(T) et droit(x) = Nil(T) Alors
    Si père(x) = Nil(T) Alors
      racine(T) <- Nil(T)
    Sinon
      Si x = gauche(père(x)) Alors
        gauche(père(x)) <- Nil(T)
      Sinon
        droit(père(x)) <- Nil(T)
    FinSi
    Si couleur(x) = Noir Alors
      père(Nil(T)) <- père(x)
      RNCorrection(T,x)
    FinSi
  FinSi
  Sinon Si gauche(x) = Nil(T) ou droit(x) = Nil(T) Alors
    Si gauche(x) <> Nil(T) Alors
      filsde_x <- gauche(x)
    Sinon
      filsde_x <- droit(x)
    FinSi
    père(filsde_x) <- père(x)
    Si père(x) = Nil(T) Alors
      racine(T) <- filsde_x
    Sinon si gauche(père(x)) = x Alors
      gauche(père(x)) <- filsde_x
    Sinon
      droit(père(x)) <- filsde_x
    FinSi
    Si couleur(x) = Noir Alors
      RNCorrection(T,filsde_x)
    Sinon
      min <- ABRMinimum(droit(x))
      clé(y) <- clé(min)
      RNSuppression(T,min)
    FinSi
  FinSi
  Retourner racine(T)
Fin

```

6. Correction

L'algorithme de correction dans un arbre rouge et noir est présenté ci-dessous. C'est le même qui est utilisé dans l'insertion comme dans la suppression.

```
RNCorrectionion(T,x)
Début
  TantQue x <> racine(T) et couleur(x) = Noir Faire
    Si x = gauche(père(x)) Alors
      w <- droit(père(x))
      Si couleur(w) = Rouge Alors
        // cas 1
        couleur(w) <- Noir
        couleur(père(w)) <- Rouge
        RotationGauche(T,père(x))
        w <- droit(père(x))
      FinSi
      Si couleur(gauche(w)) = Noir et couleur(droit(w)) = Noir Alors
        // Cas 2
        couleur(w) <- Rouge
        x <- père(x)
      Sinon Si couleur(droit(w)) = Noir Alors
        // Cas 3
        couleur(gauche(w)) <- Noir
        couleur(w) <- Rouge
        RotationDroite(T,w)
        w <- droit(père(x))
      FinSi
      // Cas 4
      couleur(w) <- couleur(père(x))
      couleur(père(x)) <- Noir
      couleur(droit(w)) <- Noir
      RotationGauche(T,père(x))
      x <- racine(T)
    Sinon
      (même chose que précédemment en échangeant droit et gauche)
    FinSi
  FinTantQue
  couleur(x) <- Noir
Fin
```

7. Complexité

❖ Théorème

Un arbre rouge et noir contenant n nœuds internes a une hauteur au plus égale à $2 \log(n + 1)$.

❖ Démonstration

Par induction, on peut montrer que le sous-barbe (d'un arbre rouge et noir) enracinée en un nœud x quelconque contient au moins $2h_n(x)$ nœuds internes, où $h_n(x)$ est la hauteur noire de x . Sachant que la hauteur est toujours inférieure au double de la hauteur noire on en déduit la borne du théorème.

❖ Conséquence

Ce théorème montre que les arbres rouge et noir sont relativement **équilibrés** : la hauteur d'un arbre rouge et noir est au pire du double de celle d'un arbre binaire parfaitement équilibré.

❖ Conclusion

Toutes les opérations sur les arbres rouge et noir sont de coût $O(h)$, c.-à-d. $O(\log n)$. Ceci justifie leur utilisation par rapport aux arbres binaires de recherche classiques

IV. Comparaison entre arbres binaires de recherche et rouge-noir

1. Comparaison

❖ Complexité temporelle de recherche

- La complexité temporelle des opérations de recherche, d'insertion et de suppression dans un ABR peut être $O(n)$ dans le pire des cas, en raison de la possibilité d'un déséquilibre de l'arbre.
- Les opérations sur les ARN garantissent une complexité temporelle de $O(\log n)$ dans le pire des cas, offrant ainsi des performances plus prévisibles et efficaces.

❖ Equilibre de l'arbre

- Les ABR ne garantissent pas d'équilibre structurel spécifique, ce qui peut conduire à des performances de recherche incohérentes dans les pires cas.
- Les ARN garantissent un équilibre relatif en respectant les propriétés de couleur et de structure, ce qui garantit des performances de recherche et de manipulation plus cohérentes.

❖ Gestion des insertions et suppressions

- Les ABR peuvent devenir déséquilibrés après des opérations fréquentes d'insertion et de suppression, nécessitant des rééquilibrages périodiques pour maintenir les performances.
- Les ARN gèrent de manière plus efficace les opérations d'insertion et de suppression en maintenant un équilibre relatif de l'arbre grâce à des ajustements de couleur et de structure.

❖ Performances dans différents scénarios

- Les ABR sont efficaces pour les petites structures de données où l'équilibre de l'arbre est moins critique, mais peuvent montrer des performances médiocres dans les grands ensembles de données.
- Les ARN sont plus adaptés aux grandes structures de données où l'équilibre et les performances de recherche prévisibles sont cruciaux, notamment dans les systèmes de gestion de bases de données et d'indexation.

En analysant ces différents aspects de comparaison entre les ABR et les ARN, il devient clair que les ARN offrent des performances plus stables et prévisibles dans une large gamme de scénarios, tandis que les ABR sont plus adaptés pour des structures de données plus petites et moins complexes.

2. Domaines d'applications

❖ Arbres binaires de recherche

- **Algorithmes de recherche** : Ils servent de base à de nombreux algorithmes de recherche, tels que la recherche binaire, utilisée pour accélérer la recherche d'éléments dans des collections triées.
- **Analyse de texte** : Les ABR sont utilisés pour la construction d'arbres de recherche de texte, notamment dans le traitement du langage naturel et l'analyse de données textuelles.

❖ Arbres rouges et noirs

- **Structures de données persistantes** : Les ARN sont utilisés dans la mise en œuvre de structures de données persistantes, ce qui signifie que les données peuvent être stockées de manière durable et récupérées même après l'arrêt ou le redémarrage d'un système informatique.
- **Systèmes de gestion de bases de données** : Ils sont utilisés pour optimiser la recherche et la récupération de données dans les systèmes de gestion de bases de données relationnelles et orientées colonnes.

CONCLUSION

En conclusion, nous avons exploré les principes fondamentaux des arbres binaires de recherche et découvert comment ils permettent une organisation efficace des données. Nous avons également étudié les arbres binaires de recherche rouge-noir, une variante qui garantit un équilibre de l'arbre et des temps d'opérations optimisés. Ces deux structures de données sont essentielles dans le domaine de l'informatique, en particulier dans les systèmes de gestion de bases de données et les recherches performantes. Les arbres binaires de recherche offrent une grande flexibilité dans la manipulation et la recherche de données, tandis que les arbres binaires de recherche rouge-noir ajoutent des propriétés supplémentaires pour améliorer l'équilibre de l'arbre. Comprendre le fonctionnement interne de ces structures de données est crucial pour optimiser les performances et les complexités des algorithmes. Enfin, il est important de noter que ces concepts peuvent être appliqués à d'autres domaines que l'informatique. Par exemple, les relations de parenté d'une famille peuvent être représentées par un arbre binaire pour faciliter les recherches généalogiques. En somme, les arbres binaires de recherche et les arbres binaires de recherche rouges noirs sont des outils puissants pour organiser et rechercher des données de manière efficace. Ils sont essentiels pour de nombreux domaines de l'informatique, mais leurs principes fondamentaux peuvent également être appliqués dans d'autres contextes.

Références bibliographiques et webographiques

[1] Support de cours par *Karine Zampieri et Stéphane Rivière*, sur **Arbres rouge et noir [rn] Algorithmique**, Unisciel algoprog, pages 1-12, [21 mai 2018] ;

[2] Support de cours par *Dr. Dhouha Maatar Razgallah*, sur **Cours Algorithmique avancée (WI)**, Ecole Supérieure d'Economie Numérique, pages 5-49, [2017-2018].

[3] **Introduction à l'algorithmique** de Thomas Cormen, Charles Leiserson, Ronald Rivest, et Clifford Stein

[4] [Arbres rouges et noirs \(irif.fr\)](http://irif.fr)

[5] <https://pierre.senellart.com/enseignement/2020-2021/cpes/cpes10.pdf>

[6] <https://www.scribd.com/document/506423313/Arbres-Rouges-Et-Noirs>